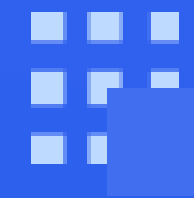




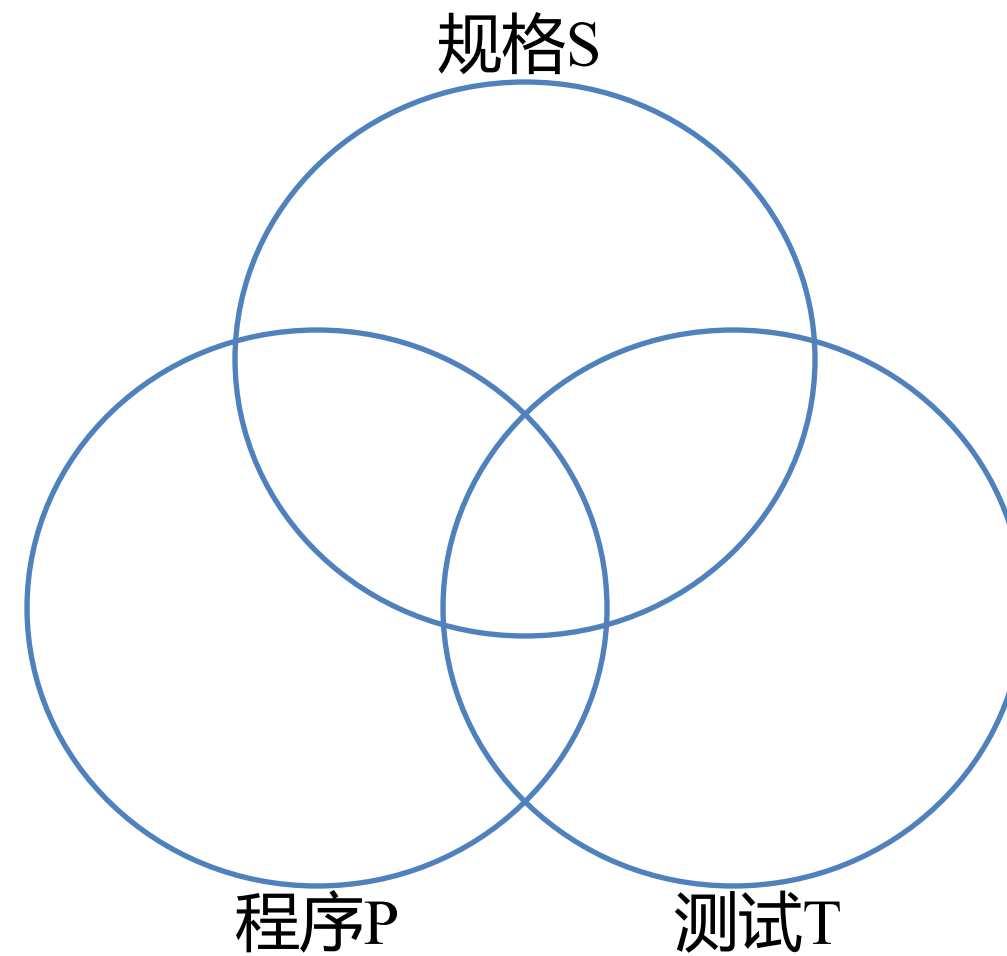
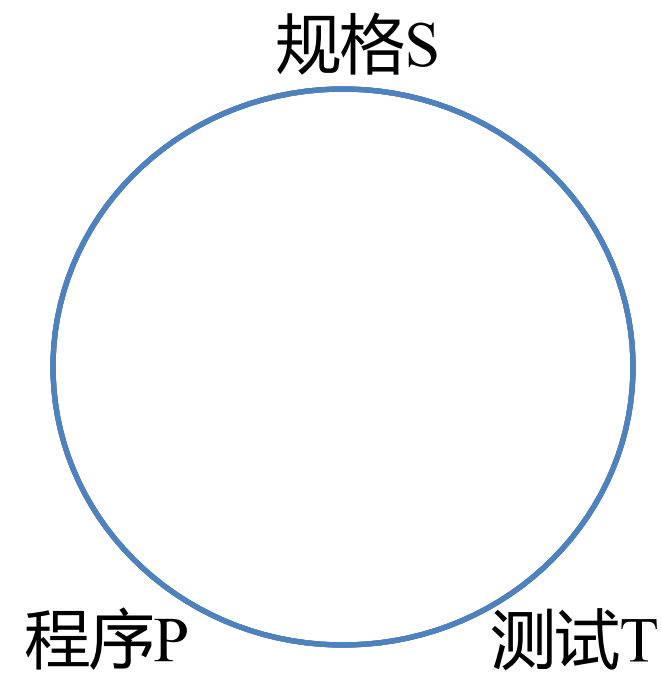
软件测试

测试框架分析



软件工程基本问题

理想 ← 鸿沟 → 现实



软件工程三大问题

- S问题：文档生成
- P问题：代码生成
- T问题：测试生成

软件质量内涵

质量定义与衡量



软件质量通常指软件与明确和隐含需求相一致的程度，是衡量软件优劣的核心指标。

给定待测程序 P 和规格需求 S ，软件质量 Q 可定义为 $Q(P, S) = P \cap S / P \cup S$ ，直观反映质量水平。

行为一致性分析



$P \cap S$ 表示程序和规格的行为一致性，是软件质量的基石，需重点关注。

需关注未被程序满足的规格需求 $(S - P)$ 和程序中超出规格需求的部分 $(P - S)$ ，以发现潜在问题。

隐含需求考量



实践中，需求规范可能不完备或不正确，隐含需求的存在对软件质量有重要影响。

工程师在开发过程中结合实际需求，识别并实现隐含需求，以提升软件的实用性和用户满意度。

三角形程序的规格与实现

规格 S :

- a, b, c 为 1 到 100 的整数, 否则输出 “无效输入值”
- a, b, c 三边相等输出 “等边三角形”
- a, b, c 两边相等输出 “等腰三角形”
- 其他情况输出 “普通三角形”

程序 P

```
def triangle(a, b, c):  
    if not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b == c:  
        print("等边三角形")  
    elif a == b or b == c or c == a:  
        print("等腰三角形")  
    else:  
        print("普通三角形")
```

- $S - P$: 程序 P 没有实现规格 S 中 a, b, c 为 1 到 100 的整数的要求
- $P - S$: 程序 P 过度实现了规格 S 中未明确要求的“任意两边之和大于第三边”的隐含需求

备注: 程序中过度实现的需求可能有利, 也可能有害。请举例说明

测试分析框架

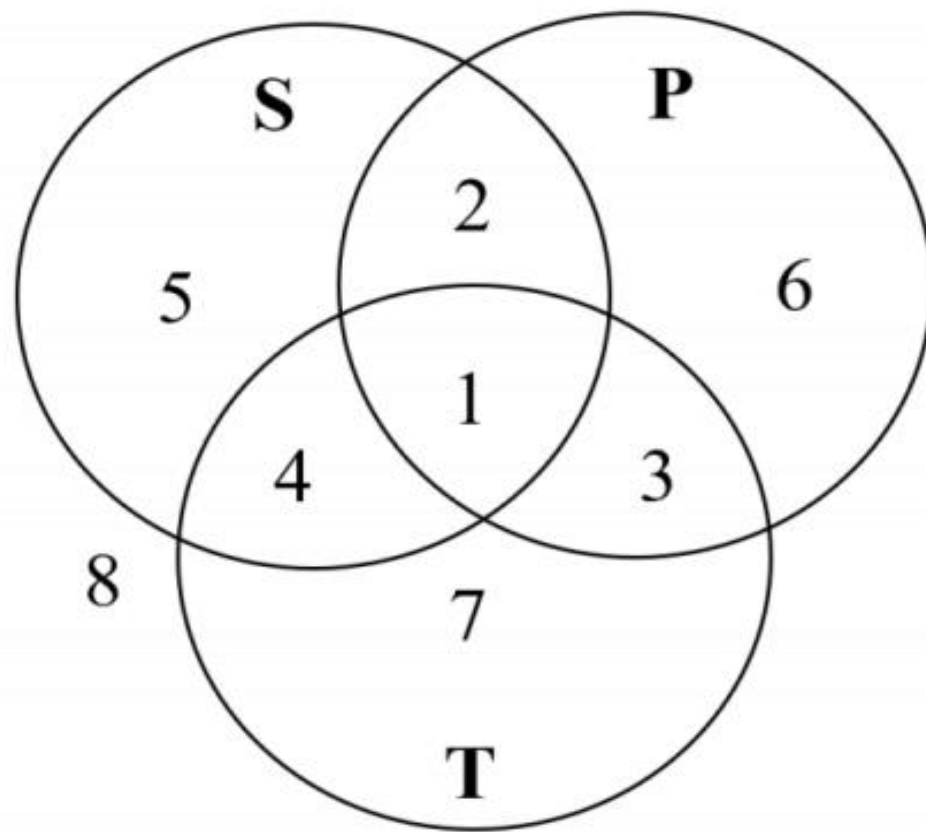


测试分析框架意义与思路

- 测试分析框架是软件测试理论和方法的基础，为测试活动提供系统化指导。
- 它假设需求规范是正确的，程序应严格遵循需求规范实现，确保软件质量。
- 需求规格 (S)、待测程序 (P) 和测试数据 (T) 三者关系紧密，需进行形式化分析。
- 软件测试理论和方法旨在指导研发新技术和工具，寻找程序特征与需求和测试的有效组合。
- 通过理论指导，优化测试策略，提高测试效率和准确性，促进软件质量提升。

测试分析框架

测试维恩图



Gourlay 测试分析框架

软件测试分析框架可被定义为多元组

$\langle \mathbb{P}, \mathbb{S}, T \rangle$ 的活动总称，其中：

1. $\mathbb{P}$ 是一组程序
2. $\mathbb{S}$ 是一组规格
3. T 是一组测试

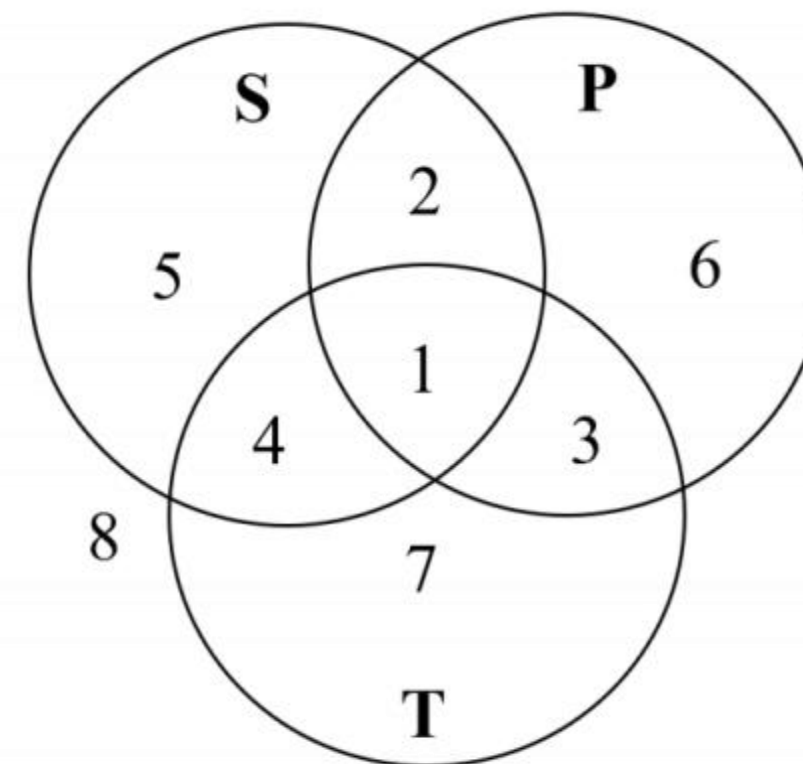
这里 T 是给定 S 和 P 后构建的测试，其中 S 和 P 分别为 $\mathbb{P}$ 和 $\mathbb{S}$ 的元素，即实例化的规格和程序。

三角形程序的规格与实现

- 区域1：同时出现在 S 、 P 和 T 的部分
- 区域2： $(S \cap P) - T$ ，未测试的指定行为
- 区域3： $T \cap (P - S)$ ，测试了未指定的行为
- 区域4： $T \cap (S - P)$ ，测试了未实现的行为
- 区域5： $S - P - T$ ，未测试的 P 未实现行为
- 区域6： $P - S - T$ ，未测试的 S 未指定行为
- 区域7：测试了既不属于 S 又不属于 P 的行为

针对前面的 S 和 P ，测试用例集 T 具有以下分析：

1. $\langle 2, 2, 2 \rangle$ ：区域1，测试了同属于 S 和 P 的行为
 2. $\langle 1, 2, 3 \rangle$ ：区域3，测试了属于 P 但不属于 S 的行为
 3. $\langle 0, 2, 2 \rangle$ ：区域4，测试了属于 S 但不属于 P 的行为
 4. $\langle 2.0, 2, 3 \rangle$ ：区域7，测试了既不属于 S 又不属于 P 的行为
- 通过测试用例集 T 可以验证程序 P 是否正确实现了规格 S

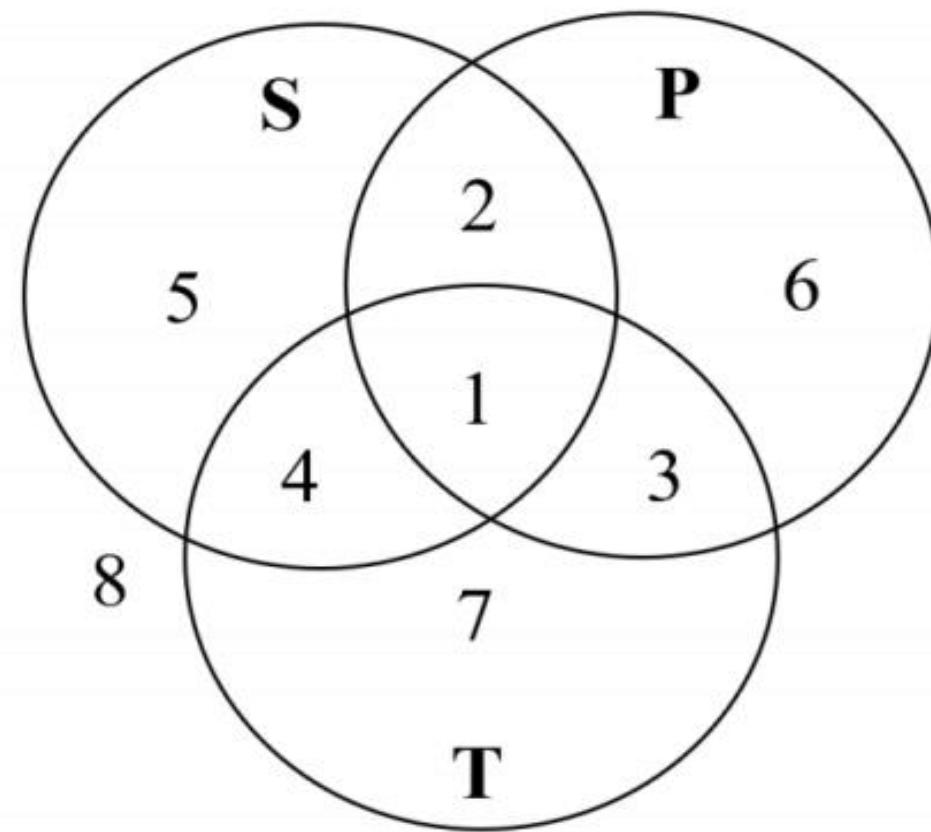


编号	区域	输入 a	输入 b	输入 c	预期输出
1	1	2	2	2	等边三角形
2	3	1	2	3	无效三角形
3	4	0	2	2	无效输入值
4	7	2.0	2	3	—

思考：尝试构建其他区域的测试用例示例

测试分析框架

测试维恩图



测试预言
 $\mathbb{O}$ 与 O

Gourlay 测试分析扩展框架

软件测试分析框架可被定义为多元组

$\langle \mathbb{P}, \mathbb{S}, T, \mathbb{O}, O \rangle$ 的活动总称，其中：

1. $\mathbb{P}$ 是一组程序
2. $\mathbb{S}$ 是一组规格
3. T 是一组测试
4. $\mathbb{O} \subseteq \mathbb{P} \times \mathbb{S}$
5. $O \subseteq \mathbb{P} \times \mathbb{S} \times T$

$\mathbb{O}$ 是通用测试预言

O 是实际测试预言

- $\mathbb{O}(P, S)$: 理论上的通用测试预言，表示程序 P 相对于规范 S 是正确的。
- $O(P, S, t)$: 实际应用中的弱化测试预言，表示在测试 t 条件下程序 P 相对于规范 S 被判断为正确。
- $\forall t \in T : O(P, S, t)$: 表示程序 P 在所有测试都符合规范 S 。
- $\neg \mathbb{O}(P, S)$: 表示程序 P 不满足规范 S ，存在缺陷； $\neg O(P, S, t)$ 类似。

测试预言的性质

对于给定规格 S ，待测程序 P ，测试 t ，

➤ **正确性：**

如果测试预言 $O(P, S, t)$ 成立，则程序 P 在测试 t 上确实执行正确 $corr(P, S, t)$ 。

➤ **完备性：**

如果程序 P 在测试 t 上执行正确 $corr(P, S, t)$ ，则测试预言 $O(P, S, t)$ 成立。

➤ **完美性：**

测试预言同时满足正确性和完备性，即 $corr(P, S, t) \Leftrightarrow O(P, S, t)$ 。

测试集T的局限性： 实际难以穷尽所有可能的测试集 T ，因此测试预言的完美性难以实现。

测试预言过于精确： 可能导致程序实际正确但测试预言不通过（例如计算精度问题）。

在软件工程中，如何平衡测试预言的正确性和完备性是一个复杂问题。

思考与讨论

测试预言的类型和构建方法?

软件测试方法的定义

测试方法 M 是一个函数: $M:P \times S \rightarrow T$,

表示基于被测程序 P 和规范 S 生成测试用例 $t \in T$ 。

生成测试集 T 的测试方法: $M:P \times S \rightarrow 2^T$

测试准则的定义

$$T_C \subseteq P \times S \times 2^T$$

表示测试准则 T_C 与待测程序 P 、规范 S 和测试集 T 的关系。

$$C(P \times S) \rightarrow 2^T$$
$$(P \times S \times 2^T) = r, \quad r \in [0, 1]$$

测试预言与测试准则

$$O_C \subseteq P \times S \times T_O$$

表示测试预言 O_C 与待测程序 P 、规范 S 和测试输出 T_O 的关系。

测试充分性可扩充定义为测试集和测试预言的配对充分性，即

$$T_C \subseteq P \times S \times 2^T \times T_O$$

测试准则蕴含的定义

$${}_2Tc_1 \subseteq {}_2Tc_2, \text{ 记为 } C_1 \geq C_2$$

表示满足 C_1 的测试集一定满足 C_2

$$C_1 \geq C_2 \text{ 且 } C_2 \geq C_1, \text{ 记为 } C_1 \equiv C_2$$

$$C_1 \geq C_2 \text{ 且 } C_1 \not\equiv C_2, \text{ 记为 } C_1 > C_2$$

偏序关系，但不是全序关系

经验关系，但不是理论关系

测试预言的包含关系

测试预言的强弱关系

定义：对于程序 P ，如 $\forall t \in T, O_1(P, t) \Rightarrow O_2(P, t)$ ，
则 O_1 比 O_2 强，记为 $O_1 \geq O_2$ 。

等价关系：如果 $O_1 \geq O_2$ 且 $O_2 \geq O_1$ ，则 $O_1 \equiv O_2$ ，
即 $\forall t \in T, O_1(P, t) \Rightarrow O_2(P, t)$

定义

测试预言能力增强

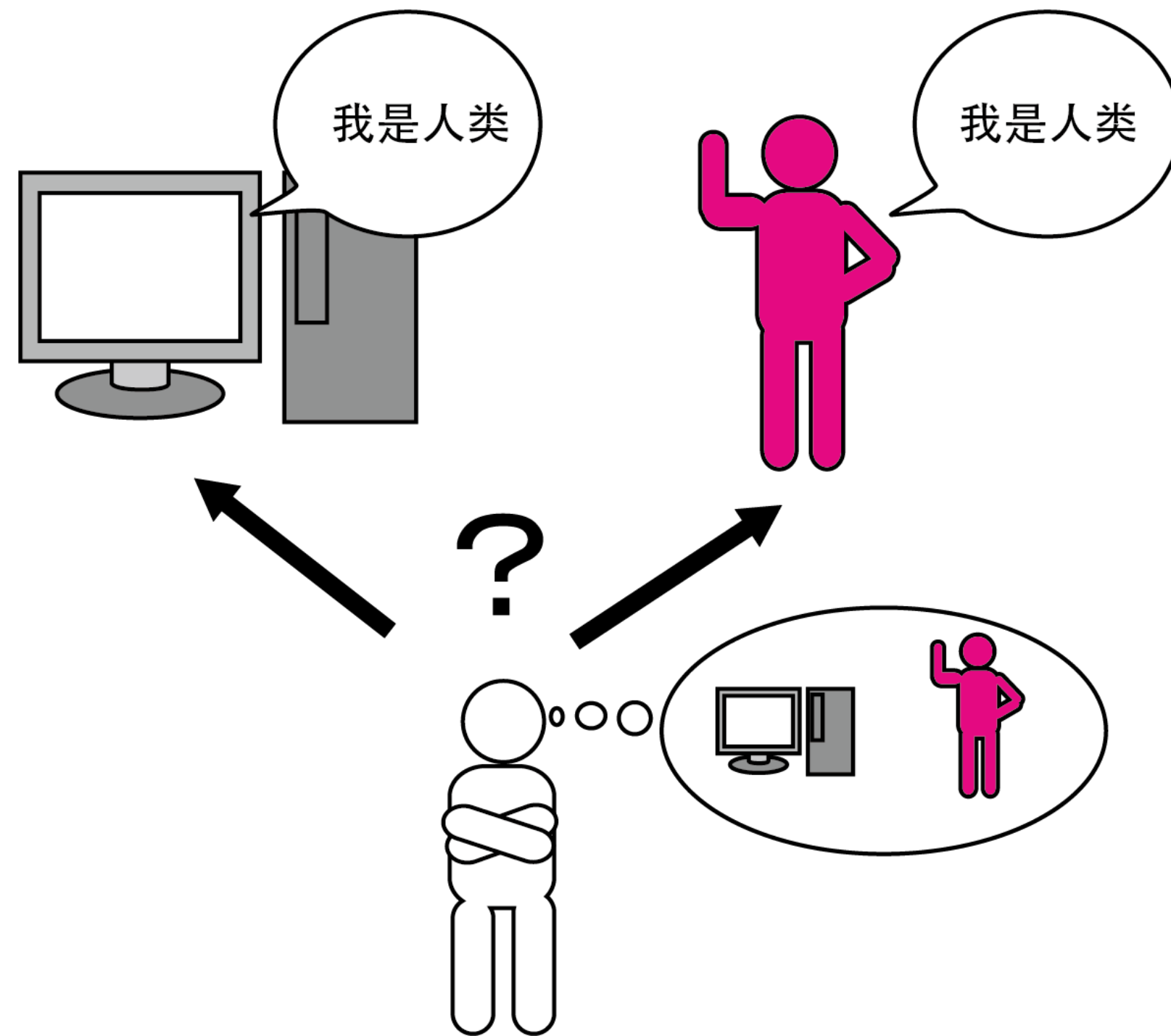
断言 A 定义的测试预言 O_A 。

附加断言 B 定义的测试预言 O_{A+B} 。

则有关系： $O_{A+B} \geq O_A$ ，因为 O_{A+B} 使用了 O_A
的超集断言，从而增强了测试预言能力。

示例

图灵测试与软件测试三大基本问题



1950 年，英国的数学家和计算机科学家艾伦·图灵在《思维》杂志上发表了一篇 名为《计算机与智能》的著名论文，这篇论文提出了一个问题：“机器能否思考？” 这个问题至今仍然是人工智能领域的一个重要问题。

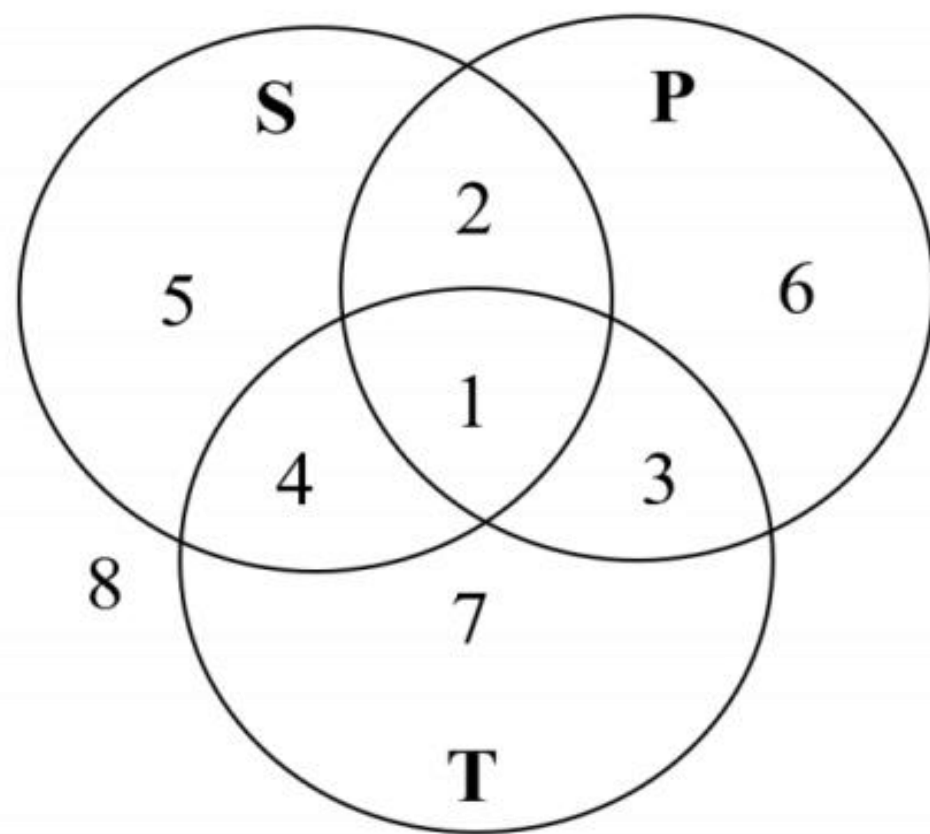


图灵测试诱导了测试的三个基本问题：

- 问题 1: 应该问什么问题？即测试生成问题。
- 问题 2: 什么是正确答案？即测试预言问题。
- 问题 3: 问多少问题够了？即测试终止问题。

回顾与讨论

测试维恩图



测试预言
 $\mathbb{O}$ 与 O

Gourlay 测试分析扩展框架

软件测试分析框架可被定义为多元组

$\langle \mathbb{P}, S, T, \mathbb{O}, O \rangle$ 的活动总称，其中：

1. $\mathbb{P}$ 是一组程序
2. S 是一组规格
3. T 是一组测试
4. $\mathbb{O} \subseteq \mathbb{P} \times S$
5. $O \subseteq \mathbb{P} \times S \times T$

- 软件测试方法
- 软件测试准则
- 测试准则蕴含
- 测试预言机制